

Costruire un bot Vinted con Node.js

Guida pratica per principianti • Versione del 8 ottobre 2026

1. Da un'idea a cinque compiti piccoli

La richiesta iniziale sembrava grande: cercare occasioni e inviare messaggi. L'ho divisa in cinque operazioni: leggere la configurazione, aprire una sessione, cercare articoli, eliminare i duplicati e notificare su Telegram. Ogni operazione diventa una funzione verificabile. Non c'è un algoritmo che scopre automaticamente il valore di mercato: il bot applica i criteri che scegli tu.

Il flusso è: avvio → configurazione → homepage e cookie → catalogo → filtro prezzo e margine → controllo ID → Telegram → registrazione ID → attesa del prossimo controllo.

La rapidità viene dall'uso di componenti già esistenti e dalla generazione assistita del codice. La parte che richiede prove è il collegamento ai servizi reali. Nel nostro caso il primo endpoint restituiva 404: una soluzione plausibile e testata con simulazioni non era ancora verificata su Vinted.

Obiettivo di questa guida: capire il programma pubblicato, ricostruirne i pezzi e saper diagnosticare un problema. I frammenti mostrati spiegano il codice; per eseguirlo usa il bot.js completo del repository, riprodotto anche nell'appendice PDF.

2. Preparare il progetto su Windows

Installa Node.js LTS 22 o successivo da <https://nodejs.org>, con Add to PATH selezionato. Chiudi e riapri il terminale. Node esegue JavaScript sul computer; npm installa le librerie.

Nel Prompt dei comandi, verifica e apri la cartella estratta:

```
node --version
npm --version
cd /d C:\Users\valep\Desktop\vinted-main
npm ci
```

Per costruire un progetto nuovo, invece, crea una cartella, esegui `npm init -y` e `npm install dotenv node-cron tough-cookie`. Imposta "type": "module" in package.json per usare import. Nel progetto già scaricato questi passi sono già stati fatti: non ripeterli.

dotenv legge .env; node-cron pianifica controlli; tough-cookie gestisce cookie e scadenze. fetch è incluso in Node moderno, quindi non serve axios. package-lock.json registra le versioni risolte; npm ci le reinstalla senza riscrivere il lockfile. node_modules contiene le dipendenze e non va caricato su GitHub.

3. Configurazione e segreti

Se non hai ancora `.env`, copialo dal modello. Se esiste e contiene i tuoi dati, non sovrascriverlo.

```
copy .env.example .env
notepad .env
```

Esempio di configurazione (le credenziali sono segnaposto):

```
TELEGRAM_BOT_TOKEN=INSERISCI_IL_TUO_TOKEN
TELEGRAM_CHAT_ID=INSERISCI_IL_TUO_CHAT_ID
VINTED_BASE_URL=https://www.vinted.it
VINTED_CATALOG_URL=https://api.vinted.it/svc-catalogue/items
SEARCH_TEXT=bracciale pandora argento
PRICE_TO=25
ORDER=newest_first
PER_PAGE=20
CRON_SCHEDULE=*/30 * * * * *
ESTIMATED_RESALE_PRICE=
MIN_PROFIT=10
ESTIMATED_COSTS=8
```

`import 'dotenv/config'` carica il file all'avvio. `process.env.SEARCH_TEXT` legge una variabile. `readConfig` converte numeri, controlla gli URL e rifiuta una configurazione incompleta. Ogni modifica richiede il riavvio del bot.

`.gitignore` esclude `.env`, `node_modules` e i log. `.env.example` contiene solo nomi e valori pubblici. Se un token viene pubblicato per errore, revocalo con BotFather: cancellarlo dall'ultimo file non lo rimuove dalla cronologia Git.

4. Trovare l'endpoint e creare una sessione

Un'API è un indirizzo che restituisce dati strutturati. La ricerca visibile sul sito effettua richieste che puoi osservare: apri Vinted, premi Ctrl+Maiusc+I, seleziona Network/Rete e Fetch/XHR, poi fai una ricerca. Cerca items e leggi Request URL e Status Code. Non condividere cookie, token, intestazioni complete o Copy as cURL.

Il vecchio `/api/v2/catalog/items` restituiva 404. La richiesta che hai osservato il 8 ottobre 2026 era `https://api.vinted.it/svc-catalogue/items` e rispondeva 200. Questo è un endpoint osservato sul sito, non un contratto pubblico garantito per sempre.

Il bot visita prima `https://www.vinted.it/`. Riceve i Set-Cookie, li salva nel CookieJar e allega alle richieste successive soltanto i cookie consentiti per quel dominio e percorso. Cookie del solo `www` non vengono automaticamente inviati ad api; un cookie `Domain=.vinted.it` può valere per entrambi. Non allarghiamo artificialmente il loro dominio.

```
const cookie = await jar.getCookieString(url);
// Dopo la risposta HTTP:
for (const value of response.headers.getSetCookie()) {
  await jar.setCookie(value, response.url || url);
}
```

Un User-Agent desktop descrive il client; non autentica l'utente e non garantisce accesso. Il bot non risolve CAPTCHA. Un 200 nel browser non garantisce un 200 da Node: il browser può avere una sessione diversa.

5. Costruire la ricerca e filtrare gli articoli

URLSearchParams codifica spazi e caratteri speciali: evita di concatenare manualmente parole chiave nell'URL. Il programma costruisce i parametri search_text, price_to, order, per_page, page e un nuovo global_search_session_id tramite randomUUID.

```
const url = new URL(config.catalogUrl);
const params = new URLSearchParams({
  search_text: config.searchText,
  price_to: String(config.priceTo),
  order: config.order,
  page: '1'
});
url.search = params.toString();
```

await fetch attende una risposta senza bloccare tutto il motore JavaScript. await response.json converte il JSON in oggetti. Prima di leggere gli articoli, getItems verifica che data.items sia un array: non considera valida una risposta con una forma sconosciuta.

Nel tuo controllo reale sono arrivati 20 articoli; price contiene amount e currency_code. Il bot converte amount in numero e applica anche localmente PRICE_TO, senza affidarsi solo al filtro remoto. Legge la prima pagina: annunci molto numerosi possono sfuggire tra due controlli.

La risposta osservata non include size_title e brand_title: il codice mostra N/D per quei campi. Non abbiamo ancora adattato taglia e marca al nuovo formato. Aggiungere campi richiede osservarne la struttura, non indovinarla.

6. Collegare Telegram

Su Telegram apri @BotFather ufficiale, invia /newbot e scegli nome e username. Copia il token in .env. Apri il tuo bot, premi Avvia e inviagli un messaggio. In locale visita <https://api.telegram.org/bot<TOKEN>/getUpdates> sostituendo <TOKEN>; cerca l'id dentro chat, non message_id né l'id del bot. L'URL contiene il token: non dividerlo. Se result è vuoto, invia un nuovo messaggio e riprova. Un webhook già configurato rende getUpdates indisponibile.

L'invio è una richiesta POST con corpo JSON:

```
const endpoint = `https://api.telegram.org/bot${config.token}/sendMessage`;
const response = await fetch(endpoint, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    chat_id: config.chatId,
    text: 'Titolo, prezzo e link dell\'articolo'
  })
});
```

Il codice completo usa un timeout, controlla lo stato HTTP e richiede ok:true nella risposta Telegram. Il messaggio è testo semplice: niente interpretazione HTML di titoli degli annunci. Il link viene costruito dall'ID sul dominio Vinted configurato.

7. Duplicati e controlli periodici

Un Set contiene valori unici. `notified.has(id)` verifica se l'articolo è già stato segnalato. `notified.add(id)` avviene soltanto dopo l'invio confermato: se Telegram fallisce, l'articolo non viene segnato come consegnato.

```
if (notified.has(id)) continue;
await notify(item, amount, currency);
notified.add(id);
```

Il Set è in memoria, si azzerà al riavvio e cresce nel tempo. Non garantisce consegna esattamente una volta: Telegram potrebbe accettare un messaggio e la risposta potrebbe andare persa, causando un duplicato al nuovo tentativo. Al primo avvio il bot segnala anche gli annunci già presenti e compatibili.

node-cron usa sei campi: secondi, minuti, ore, giorno del mese, mese, giorno della settimana. `*/30` * * * * * controlla ai secondi 0 e 30; `*/10` * * * * * ogni 10 secondi; `0` * * * * * ogni minuto. Una frequenza maggiore non garantisce più profitto e può portare a limiti 429.

running evita due controlli contemporanei. `nextCheckAt` sospende le nuove richieste durante un'attesa dopo errore. Il primo controllo parte subito, poi cron richiama check. Lo schermo può spegnersi; sospensione, ibernazione o spegnimento fermano l'esecuzione. Un processo locale non diventa automaticamente un servizio sempre acceso.

8. Gestire errori e provare il programma

401: il bot elimina i vecchi cookie, visita di nuovo la homepage e ritenta la ricerca una volta. 403: accesso negato; non è un invito ad aggirare una verifica. 404: indirizzo non disponibile o risposta di accesso; si indaga prima di cambiare l'URL. 429: troppe richieste; il bot considera Retry-After e applica un'attesa minima di 60 secondi.

Gli errori consecutivi producono attese progressive. Con l'implementazione attuale la sequenza base è 60, 120, 240, 480, 960, 1800 secondi e poi resta a 1800; Retry-After può imporre un'attesa più lunga. Dopo un controllo riuscito il contatore si azzerà. Ogni richiesta ha un timeout di 20 secondi.

Esegui nell'ordine:

```
npm test
node bot.js --check
npm start
```

I sei test simulati coprono diagnostica senza Telegram, risposta inattesa, rinnovo 401 e deduplicazione, filtri, errore Telegram 429 e assenza di sovrapposizioni. In `createMonitor(config, fetchFn)` sostituiamo `fetch` con risposte controllate: i test non inviano messaggi reali.

`--check` verifica l'accesso reale al catalogo e mostra conteggi e nomi di campi, senza inviare notifiche. Nel tuo PC ha restituito 20 articoli. `npm start` verifica il flusso completo; hai poi confermato il funzionamento. I test simulati non dimostrano da soli accesso a Vinted, consegna Telegram o esattezza di una stima economica.

9. Calcolare il margine e migliorare il bot

Margine stimato = prezzo realistico di rivendita - prezzo articolo - costi stimati. Esempio puramente didattico: $45 - 25 - 8 = 12$ euro. Con `ESTIMATED_RESALE_PRICE=45`, `ESTIMATED_COSTS=8` e `MIN_PROFIT=12`, un articolo da 25 euro passa il filtro. La soglia di rivendita vale per tutta la ricerca e il filtro accetta soltanto EUR: non è adatta a lotti molto diversi tra loro.

Se lasci `ESTIMATED_RESALE_PRICE` vuoto, il filtro sul margine è disattivato: il bot controlla solo il prezzo massimo. Stime, autenticità, condizioni e domanda restano responsabilità della valutazione umana. Il programma non compra articoli.

Prossimi miglioramenti sensati: salvare gli ID in un archivio locale con scadenza; supportare più ricerche; leggere marca e taglia dal nuovo formato; mostrare un riepilogo dei controlli; aggiungere test per Retry-After e errori di rete. Sono idee, non funzioni già implementate. Prima di ogni modifica scegli quale comportamento verificare.

10. Il metodo da riutilizzare

1. Scrivi input e risultato atteso: ricerca, soglia prezzo, messaggio. 2. Dividi in funzioni piccole con una responsabilità ciascuna. 3. Usa librerie mantenute per compiti comuni, senza ricostruire cookie e cron. 4. Verifica il servizio reale con una richiesta non distruttiva. 5. Scrivi test per errori e comportamenti importanti, non solo per il caso felice. 6. Tieni separati configurazione pubblica e credenziali. 7. Pubblica soltanto i file necessari e documenta cosa è ancora incerto.

L'errore iniziale ci insegna una cosa concreta: l'endpoint e il formato dei dati devono essere confermati prima di chiamare il sistema pronto. Il browser ha fornito un indizio; la diagnostica sul tuo PC ha verificato l'accesso del bot.

Documentazione: <https://nodejs.org/en/learn> • <https://nodecron.com> • <https://core.telegram.org/bots/api> • <https://github.com/motdotla/dotenv> • <https://github.com/salesforce/tough-cookie>

Repository e codice completo: <https://github.com/ValePrugna/vinted>

Appendice • Il codice completo

Copia eseguibile di bot.js alla versione di questa guida. Usa il file del repository per evitare errori di trascrizione. Le righe lunghe sono mandate a capo soltanto per la stampa.

```
1 import 'dotenv/config';
2 import cron from 'node-cron';
3 import { CookieJar } from 'tough-cookie';
4 import { pathToFileURL } from 'node:url';
5 import { randomUUID } from 'node:crypto';
6
7 const USER_AGENT = 'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/131.0.0.0 Safari/537.36';
8
9 export function readConfig(env = process.env, requireTelegram = true) {
10   const number = (name, fallback) => {
11     const value = env[name]?.trim() ? Number(env[name]) : fallback;
12     if (!Number.isFinite(value) || value < 0) throw new Error(`${name}: numero non negativo
  richiesto`);
13     return value;
14   };
15   const base = new URL(env.VINTED_BASE_URL || 'https://www.vinted.it');
16   if (base.protocol !== 'https:' || base.username || base.password) throw new
  Error('VINTED_BASE_URL deve essere un URL HTTPS senza credenziali');
17   // Il catalogo osservato sul sito italiano usa un host API separato.
18   const catalog = new URL(env.VINTED_CATALOG_URL || `https://${base.hostname.replace(/^www\./,
  'api.')}svc-catalogue/items`);
19   if (catalog.protocol !== 'https:' || catalog.username || catalog.password || catalog.search ||
  catalog.hash) throw new Error('VINTED_CATALOG_URL deve essere un URL HTTPS senza credenziali o
  parametri');
20   if (catalog.hostname.replace(/^www|api\./, '') !== base.hostname.replace(/^www|api\./, ''))
  throw new Error('Il catalogo deve appartenere allo stesso dominio Vinted della homepage');
21   if (requireTelegram && (!env.TELEGRAM_BOT_TOKEN || !env.TELEGRAM_CHAT_ID)) throw new
  Error('Configura TELEGRAM_BOT_TOKEN e TELEGRAM_CHAT_ID in .env');
22   const schedule = env.CRON_SCHEDULE || '*/*30 * * * * *';
23   if (!cron.validate(schedule)) throw new Error('CRON_SCHEDULE non valido');
24   const perPage = number('PER_PAGE', 20);
25   if (!Number.isInteger(perPage) || perPage < 1 || perPage > 96) throw new Error('PER_PAGE deve
  essere un intero tra 1 e 96');
26   return {
27     baseUrl: base.origin, catalogUrl: catalog.href, token: env.TELEGRAM_BOT_TOKEN, chatId:
  env.TELEGRAM_CHAT_ID,
28     searchText: env.SEARCH_TEXT || '', priceTo: number('PRICE_TO', 40),
29     order: env.ORDER || 'newest_first', perPage, schedule,
30     resalePrice: env.ESTIMATED_RESALE_PRICE?.trim() ? number('ESTIMATED_RESALE_PRICE', 0) : null,
31     minProfit: number('MIN_PROFIT', 10), costs: number('ESTIMATED_COSTS', 8),
32   };
33 }
34
35 // Dipendenza fetch iniettabile per verificare il bot senza servizi reali.
36 export function createMonitor(config, fetchFn = fetch) {
37   const jar = new CookieJar();
38   const notified = new Set(); // Si azzerà al riavvio; non contiene dati sensibili.
39   let sessionReady = false;
40   let running = false;
41   let nextCheckAt = 0;
42   let failures = 0;
43
44   async function vintedRequest(url, accept) {
45     const cookie = await jar.getCookieString(url);
46     const response = await fetchFn(url, {
47       headers: { 'User-Agent': USER_AGENT, Accept: accept, 'Accept-Language': 'it-IT,it;q=0.9',
  Referer: `${config.baseUrl}/`, ...(cookie ? { Cookie: cookie } : {}) },
48       signal: AbortSignal.timeout(20_000),
49     });
  }
```

```

50 // CookieJar rispetta dominio, percorso e scadenza dei cookie.
51 for (const value of response.headers.getSetCookie()) await jar.setCookie(value, response.url ||
url);
52 return response;
53 }
54
55 function httpError(response, service) {
56   const error = new Error(`${service}: HTTP ${response.status}`);
57   const retry = response.headers.get('retry-after');
58   const seconds = retry ? Number(retry) : NaN;
59   const delay = Number.isFinite(seconds) ? seconds * 1000 : Date.parse(retry) - Date.now();
60   if (response.status === 429) error.retryAfter = Math.max(60_000, Number.isFinite(delay) ? delay
: 60_000);
61   return error;
62 }
63
64 async function refreshSession() {
65   sessionReady = false;
66   await jar.removeAllCookies();
67   const response = await vintedRequest(`${config.baseUrl}/`, 'text/html');
68   if (!response.ok) throw httpError(response, 'Sessione Vinted');
69   await response.arrayBuffer(); // Consuma la risposta prima delle richieste successive.
70   sessionReady = true;
71 }
72
73 async function getItem() {
74   if (!sessionReady) await refreshSession();
75   const url = new URL(config.catalogUrl);
76   url.search = new URLSearchParams({ search_text: config.searchText, price_to:
String(config.priceTo), order: config.order, per_page: String(config.perPage), page: '1',
global_search_session_id: randomUUID() }).toString();
77   let response = await vintedRequest(url.href, 'application/json');
78   if (response.status === 401) {
79     await response.arrayBuffer();
80     await refreshSession();
81     response = await vintedRequest(url.href, 'application/json'); // Un solo tentativo
aggiuntivo.
82   }
83   if (!response.ok) throw httpError(response, 'Ricerca Vinted');
84   const data = await response.json();
85   if (!Array.isArray(data.items)) throw new Error('Risposta Vinted inattesa: items assente');
86   return data.items;
87 }
88
89 async function notify(item, amount, currency) {
90   // URL ricostruito sul dominio configurato, senza fidarsi di link esterni nella risposta.
91   const link = new URL(`/items/${item.id}`, config.baseUrl).href;
92   const profit = config.resalePrice === null ? '' : `Margine stimato: ${config.resalePrice -
amount - config.costs}.toFixed(2)} EUR`;
93   const message = `Titolo: ${String(item.title || 'N/D').slice(0, 600)}\nPrezzo:
${amount.toFixed(2)} ${currency}\nTaglia: ${String(item.size_title || 'N/D').slice(0, 100)}\nMarca:
${String(item.brand_title || 'N/D').slice(0, 100)}${profit}\nLink: ${link}`;
94   const response = await fetchFn(`https://api.telegram.org/bot${config.token}/sendMessage`, {
95     method: 'POST', headers: { 'Content-Type': 'application/json' },
96     body: JSON.stringify({ chat_id: config.chatId, text: message, link_preview_options: {
is_disabled: true } }),
97     signal: AbortSignal.timeout(20_000),
98   });
99   if (!response.ok) throw httpError(response, 'Telegram');
100   const data = await response.json();

```

```

101     if (data.ok !== true) throw new Error('Telegram: invio non confermato');
102 }
103
104 async function check() {
105     // Evita sovrapposizioni se rete o notifiche richiedono più dell'intervallo cron.
106     if (running || Date.now() < nextCheckAt) return;
107     running = true;
108     try {
109         const items = await getItems();
110         for (const item of items) {
111             const id = String(item.id ?? '');
112             if (!/^d+$/.test(id) || notified.has(id)) continue;
113             const raw = typeof item.price === 'object' ? item.price?.amount : item.price;
114             const amount = raw === null || raw === undefined || raw === '' ? NaN : Number(raw);
115             const currency = item.price?.currency_code || item.currency || 'EUR';
116             if (!Number.isFinite(amount) || amount < 0 || amount > config.priceTo) continue;
117             if (config.resalePrice !== null && (currency !== 'EUR' || config.resalePrice - amount -
config.costs < config.minProfit)) continue;
118             await notify(item, amount, currency);
119             notified.add(id); // Registra soltanto invii confermati: un errore sarà ritentato.
120             console.log(`Articolo ${id}: notifica inviata`);
121         }
122         failures = 0;
123     } catch (error) {
124         failures += 1;
125         const delay = Math.max(error.retryAfter || 0, Math.min(30 * 60_000, 30_000 * 2 **
Math.min(failures, 6)));
126         nextCheckAt = Date.now() + delay;
127         // Non stampare URL Telegram o errori fetch che potrebbero contenere il token.
128         console.error('Controllo fallito (${error.message.includes('HTTP ') ? error.message : 'errore
di rete o risposta inattesa'}). Nuovo tentativo tra ${Math.ceil(delay / 1000)} secondi.`);
129     } finally {
130         running = false;
131     }
132 }
133 // Diagnostica senza Telegram: espone solo conteggi e nomi dei campi.
134 async function inspect() {
135     const items = await getItems();
136     const first = items[0];
137     return {
138         items: items.length,
139         fields: first ? Object.keys(first) : [],
140         priceFields: first && typeof first.price === 'object' && first.price !== null ?
Object.keys(first.price) : [],
141     };
142 }
143 return { check, notified, inspect };
144 }
145
146 if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
147     try {
148         const diagnostic = process.argv.includes('--check');
149         const config = readConfig(process.env, !diagnostic);
150         const monitor = createMonitor(config);
151         if (diagnostic) {
152             const result = await monitor.inspect();
153             console.log(`Catalogo raggiunto. Articoli restituiti: ${result.items}`);
154             console.log(`Campi articolo: ${result.fields.join(', ')} || '(nessun articolo)';`);
155             console.log(`Campi prezzo: ${result.priceFields.join(', ')} || '(prezzo semplice o
assente)';`);

```

```

156     } else {
157         const task = cron.schedule(config.schedule, monitor.check);
158         console.log(`Monitoraggio avviato: ${config.searchText}, prezzo massimo ${config.priceTo}`);
159         void monitor.check();
160         for (const signal of ['SIGINT', 'SIGTERM']) process.on(signal, () => { task.stop();
process.exit(0); });
161     }
162 } catch (error) {
163     // Errori fetch possono includere URL: non stampare dettagli di rete sensibili.
164     console.error(error instanceof TypeError ? 'Richiesta fallita: verifica connessione e accesso
ai domini Vinted.' : error.message);
165     process.exitCode = 1;
166 }
167 }

```